Substrates Vision Statement

Joel Jakubovic

Summary

What is (and isn't) a substrate? Ontology of state with an authoring tool (PLs automate the authoring).

What values guide our research? Design well for humans first, optimise performance later. Ultra-convenience to better colonise the minds of interested parties.

What are the potential benefits of substrates? Common language to speak; solve coordination problem. Privately (or in small group) explore consequences of a design.

What can we learn from past attempts? Have a survival/replication plan; plenty of elegant organisms went extinct. Respect Worse-is-Better Darwinism.

What are the research problems? Just how many different substrates are there, really? (Modulo different jargon or byte encodings.) What is there, which is meaningfully distinct from the ubiquitous graph of key-value dictionaries? Perhaps substrates differ mainly by graphical interface rather than the underlying data model. Is there a Holy Grail / stem-cell meta-substrate, which we could easily adapt for any use case? Or would that be too good to be true?

What do we disagree about? Whether the extinctions of nice substrates like HyperCard were just unlucky, or robust to historical perturbations. If the latter, we should be careful not to make the same mistakes.

What research methodologies should we use? I regret not studying evolutionary biology. It seems highly relevant, so I'm catching up. Discover the genuine design tradeoffs rather than cheering "simplicity" etc. On the other hand, identify low-hanging fruit with an argument for why it's not yet been picked.

What I'd like to see come out of this workshop: Interest or critique of new fields of study, esp software-convention-ology (and a better name?)

Details of my personal research vision: I'm with Kell; let's break down the walls between all the existing silos and bridge all the superficially incompatible implementations of the key/value dictionary graph. A novel substrate is unlikely to make a dent, while the heap memory of every application is *half-way* to being a lingua franca — it just needs suitable annotations and de-duplication.

1 Introduction

The topic of this workshop is software substrates. How does a substrate differ from a programming system/language? Perhaps programming systems/languages are general-purpose and Turing-complete by default. Substrates, in contrast, can be expected to be specialised for specific domains (e.g. spreadsheets) or to not have a Turing machine lurking within them.

The term "substrate" seems downstream of the view of software as a material or medium. If it is so, then we can work with the material manually; I'm tempted to dismiss "programming" as a specialised field concerned with automating that work. We automate the work that we don't want to do, but a *medium* seems like the domain of work that we *do* want to do ourselves. E.g: communicating or presenting something (limited by the audience's speed of understanding), or creating / testing some ideas (limited by the author's speed of understanding).

Software as a medium is so general that it is difficult to define in a helpful way. When in doubt, I always return to the conceptual model of analytical mechanics [10]: a physical system has a *state;* a large system might have a state composed of many parameters, and it has as many subsystems as there are subsets of parameters. These parameters, typically real numbers, are treated as primitive: nobody cares to define a "decomposition of a real number", even though possibilities exist (e.g. analysing each decimal digit or binary bit as a primitive parameter), because this is not useful in studying the natural world.

Given a specific state, we can imagine the same state but with a different value for one parameter. Then we can imagine the system jumping from the old state to the new state. There is a combinatorial explosion of such deltas, and if we accumulate them, we get logically conceivable evolution paths of the system. In natural science, we take observed empirical reality as our guidepost and ignore most of these logically possible paths in favour of the narrow range of paths that real systems actually take. The job of mechanics is to characterise which criteria nature uses to determine how systems evolve (e.g. the Principle of Least Action [4]).

In software, we begin from essentially the same premise. However, unlike the study of nature, we have vastly more freedom in how the system will evolve over time. Hence Alan Kay's view of the software medium as a meta-medium, the universal simulator: software can act like anything we want, at least relative to the capabilities of physical output devices to which it is connected. Our evaluation method for software is not whether the evolution happens in nature, but whether the evolution is useful or desired by a human being in a particular context. This subjectivity is inherent to the field; there are many people with many different goals in many different use cases. We can't expect to circumscribe it.

Of course, one common way to respond to such complexity is to ignore it: to create software that is the way it is, and mandate that users[1] adapt themselves to it. This is the path of least resistance in industry and in practices set by industry. However, I expect that we are all unimpressed with this state of affairs. Presumably, we seek substrates that are general and flexible enough to adapt to user desires, or seek specific substrates well-suited to certain domains — we see complexity of implementation as a worthy price to pay for ease of *use*.

2 Substrate = Primitives + Compositions + Editor + API

In software, as in mechanics, we can imagine that one of the primitive parameters gets changed to a new value. The delta from the old to the new state is a smallest possible "edit", which could in principle be performed manually. Any programming "language" on top of this is just a way of automating these changes and making them conditional on other parts of the state. Thus I am inclined to see the *ontology of state* ("what is state made of?") as primary, and the "language of evolution" as of secondary importance. *Given* an ontology of state, primitive edits to that state fall out quite easily, and an infinite variety of programming languages can be dreamt up which compile down to those primitive edits.

For example, in the "machine" view of digital material, state consists of one long array of machine words (or bytes, or bits). A primitive edit consists of changing one of these bytes. Machine instructions on the CPU then make these primitive edits automatically, and a great diversity of programming languages are built on top of them. However, the MS-DOS program `DEBUG.COM` also allowed a user to make these primitive changes *manually,* and GNU Poke[2] possibly does the same thing for the modern Unix process.

This leads me to offer the following definition of a substrate: it is a set of primitives, and ways to compose them, which is effectively authorable, and which has an API (i.e. it is also authorable by computer programs).

"Effectively authorable" means: a user can make primitive edits to the substrate, evolving it from one valid state to another valid state. This is relevant when we cannot afford to think of the substrate as an abstract thing, but must instead implement it for real. In order to do that, we need to implement it atop some existing lower-level substrate. If an editing interface is available for the lower-level substrate,

[1] Remember here that programmers are the users of programming systems; I remain skeptical that programmers and users have conflicting interests in general. If one's job security depends on mastery of artificial difficulty, does one have an interest in keeping it difficult, or making it easier?

[2] https://www.jemarch.net/poke

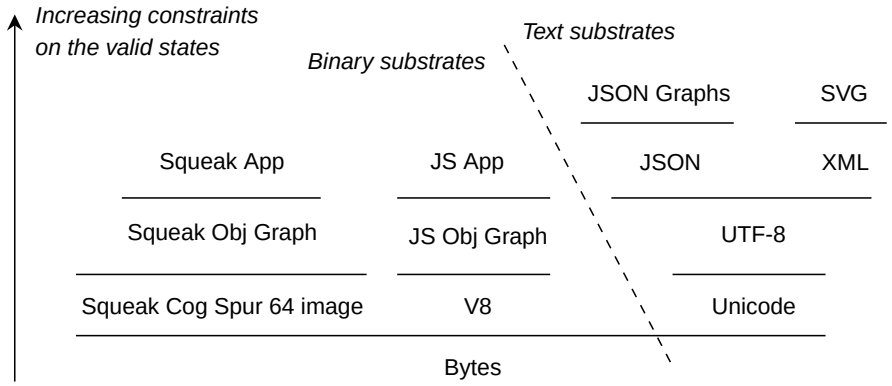

Figure 1: Sketch of some hierarchies of substrates in software. Everything builds on top of bytes, but there is an early speciation into "plain text" substrates. Each layer represents some restriction of the variations permitted by the layer below.

then there is a risk of edits that result in a state valid in the lower-level substrate, yet invalid in the higher-level one. I will illustrate this definition with the following examples.

3 Substrate Examples

"Plain text" is a low-level substrate sitting atop the even lower-level substrate of bytes. The rules of Unicode and UTF-8 constrain the allowable states of the system: some byte sequences may not correspond to a valid character. A hex editor thus allows the state of a text file to be changed to something that is not a text file, while a text editor's edits are closed within the set of valid text files. Because text editors are ubiquitous, we can class "plain text" as effectively authorable, and hence a substrate. The primitives are characters and the composition mechanism is a single flat array.

Bytes and text are two root[3] substrates on which all others are built by further restricting the valid states. Every additional constraint defines a further data format, which may fall short of being a substrate by not being effectively authorable. A programming language syntax restricts the text substrate to syntactically valid programs. Widely available editor feedback for syntax errors means that, e.g. Python syntax is a substrate. However, many syntactically valid programs are still semantically invalid programs; this suggests that from a *semantic* perspective, Python source code is not an effectively authorable substrate. (Perhaps no Turing-complete programming language is effectively authorable as so defined, because no editor could guarantee the absence of run-time errors. Is Haskell nearly a counterexample?)

A better example here is **JSON:** while editors can ensure that the file remains valid JSON, JSON is probably being used to encode some more constrained domain-specific structure with its own rules of validity (its "schema"). For example, the `age` field must always have a nonnegative integer value. By default, the JSON-aware text editor does not know what these rules are; in this case, the domain-specific data structures would not be effectively authorable and would not constitute a substrate.

Running parallel to the stack of text formats is the stack built on the non-textual ("binary") base substrate. The substrate of raw bytes is effectively authorable (vacuously so, since all byte sequences are valid) via the hex editor, but each **binary format** does not necessarily come with an editor that respects its rules. Each binary format has its own "quantitative syntax" [6] in terms of offsets and sizes, which restricts the set of valid states. There is a delicate relationship between different parts of the state, which will be broken by the insertion or deletion of a single byte; this contrasts heavily with text formats which are designed to be fairly robust to insertion/deletion. Yet in order to be useful, there clearly needs to be room for variation within the format: for example, the PNG format permits

[3]Of course, binary is the true root and text is built on top of it. It's still useful to think of them as two roots, because their descendants are disjoint in many ways. Text vs. binary data is more than a difference in alphabets; the big difference seems to be quantitative vs. qualitative syntax.

many different arrangements of coloured pixels. Because PNG editors are ubiquitous, PNG counts as a substrate.

The primitives of a binary substrate are individual bytes, machine words or fixed-size blocks; the composition mechanisms are arrays, nesting, and graph linkage, achieved by relative or absolute offsetting schemes. For example, a **Squeak Smalltalk VM image** is a binary file that contains an encoded object graph. This object graph is authorable using the Squeak VM.[4] Further data structures and applications are created as subtrees and subgraphs, with their own validity rules. Whether or not *these* class as substrates depends on whether authoring tools are provided as part of the application within Squeak — the built-in Squeak inspection and debugging tools could be used to break these application structures (while still remaining valid Squeak object graphs), so such tools are not sufficient. Similarly, the heap of a Unix process is a graph of byte blocks containing pointers to each other, though it is not necessarily a structure observable as a file on disk.

The **web page** perhaps contains two substrates: the "structural" DOM and the "behavioural" JS object graph. The DOM is effectively authorable via the browser dev tools, while the JS object graph is crudely authorable via the console. However, the changes thus made are not persistent; this should probably be part of the definition of "effectively authorable".

A contrast with **HyperCard** (something we can all surely agree was a substrate with good qualities) reveals further refinements to my definition: the web page is nothing like HyperCard because even its most basic elements (static text and graphics) cannot be directly manipulated in the page; edits must be made to its shadowy DOM tree instead. Alan Kay's criticism of the web is that it didn't bundle such an obvious authoring tool with the browser. A substrate must be writeable at a similar level of skill as it is readable.

(HyperCard may have been nice, but it was dependent on Apple and it went extinct. If this was due to a business decision made by Steve Jobs because of his unique personality, then we could consider HyperCard to have merely been unlucky, and suppose that something like it it could survive and thrive once again given an initial kick. On the other hand, if it vanished because of a sudden change in environment (e.g. competition with the Web) or due to some other robust historical fact, then this should give us caution. I would like to see explicit arguments on random chance vs. systematic factors on discussions of substrate extinction, instead of having them remain implicit assumptions. A respect for something like Darwinism is needed, as explored in Gabriel's Worse-is-Better series [5] and Kell's Unix/Smalltalk comparison [8]; substrates that are more than research curiosities deserve some thought put into a survival-and-replication plan.)

Spreadsheets have numbers, text strings, and fill colours as their primitives. The composition mechanism is a flat 2D grid. There is also nesting via drawing thick borders around things, but this not so easy to manipulate programmatically; it's more of a cue for the eyes. Perhaps the formula relations could be seen as a composition mechanism.

Normal applications have a failed substrate of **bytes in the heap;** there is no normal way to edit heap structures in a running process. If there were a format describing the heap structures (similar to what Kaitai Struct[5] does for binary files; see also liballocs [7], which aims to improvise such a format from existing debug metadata) then one could conceivably feed it as a template into a hex editor, and edit valid structures into other valid structures. But in practice, the only read/write interface to heap data is the application's user interface, many times indirected from it and heavily restricted in what it can do.

Mobile apps feel even more closed / locked-down than this. Something with very few degrees of freedom — e.g. a preferences pane in an app, with some sliders and drop-downs for specific high-level parameters of the user experience — may well be part of a useful tool for something, but it is not a substrate. A *substrate* is etymologically a *stratum* (layer) that sits *sub* (under) something, supporting a variety of combinations of its primitive elements.

[4] Because Squeak is maintained for Windows, Mac and Linux and is easy to install, I see it as *effectively* authorable. This contrasts with Self, which seems quite complicated to set up and build/run; Self images are not effectively authorable for normal people. However, if I end up succeeding at exposing the Self object graph via a FUSE filesystem interface, then it will at least become effectively browseable; a step towards a substrate.

[5] https://kaitai.io/

4 Research Values

It might be good *for morale* to unite around words like "simplicity", "elegance", "human-friendliness" etc, but serious research needs a purely descriptive, non-moral account of concepts like these. If the things we want are so good, why do we not have them already? I would like to see explicit arguments (by no means exhaustive, just an outline) that yes, there is low-hanging fruit ... and it has not been picked because X. As we move out to the Pareto frontier, i.e. the place where we have to make tradeoffs, I would like to see analysis of the inconvenient decisions that reality forces us to make.

That said, those of us in this workshop probably find certain tradeoffs untroubling. For example, I see mainstream practice as prioritising ease of implementation, at the cost of the user having to adapt himself to the software. This is partly because of the incentives and economics of industry, but also because we genuinely don't know how to make open, convivial, malleable software — it's a hard research problem. This means that making the opposite tradeoff — being willing to wrestle with difficult questions of design and implementation ourselves, over research timescales rather than business cycles — for the sake of openness / malleability, is a meaningful activity rather than a mere rallying flag.

Another example: malleability is often sacrificed for the sake of efficiency or performance. This may be necessary under the constraints of industry, but I expect we're sympathetic to the opposite tradeoff: we'd prefer to prioritise human suitability first, on the assumption that anything can be optimised later by throwing enough engineering brainpower at the problem (see Self [3], for example). At Xerox PARC, they "designed for the hardware of tomorrow" — even if hardware performance stalls, I think it is still a safe assumption that any elegant design can be optimised with enough dedication.

However, those are the only two tradeoffs I can think of that my value system classes as "easy wins".

One manifestation of "implementation effort" is in making it as easy as possible for an interested party to try out a substrate, understand its design, or contribute to it. I think it is worth considering *ultra-convenience* as an important value in today's attention-saturating environment.

5 Ultra-convenience

I personally dislike installing things: if it doesn't run in some web page, it doesn't exist. I also dislike learning a system by trial and error: if there's no readable or watchable outline of how the thing works, I have better things to do than take a random walk through the state space. Call me lazy, but it is probably a wise practical assumption that all attention will come from 2 dedicated true believers, a handful of their friends, and 10,000 easily distracted or busy people. This suggests value in studying memetics, or how ideas spread and mutate across people's minds. Even "trivial inconveniences" [1] can have a disproportionate effect on how many people engage with your prototype. Therefore, consider optimising for ultra-convenience, or virality.

Before my time, magazines listed BASIC source code of simple games that one could type in, play, and modify. The modern equivalent might be where your audience can just copy some code into the JS console in a new tab. No install, no setup, works everywhere. I'm suggesting an extreme end of the tradeoff: be willing to work harder so as to require no "virtue" from the user at all.

I see programming as a way to test interesting ideas by forcing them to take on a precise, executable form. However, it doesn't feel sensible to do this with with half-baked ideas. I prefer to discuss half-baked ideas with other people until they get to a form where the discussion can no longer help me; where the thing I need to find out cannot be proven or simulated by any person, and it can only be discovered through experiment. I prefer to read and write English prose with diagrams, or to watch an expert using a system they know well, than to prematurely write code that vastly fewer people will download. I admit, this is partly a problem of media: my wordpress.com blog template doesn't easily provide for interactive JS demos, but a future Github site might. Nevertheless, this is a nontrivial technical investment that I have yet to make. I also suppose that if we did have a malleable "personal dynamic medium" that was as convenient as pen-and-paper or communicating with people, then I might do more with it instead.

6 Fragmentation and Convention-ology

There may be value in creating a new substrate, but what are the costs? If the substrate offers something genuinely novel, then it has clear value. If it is intended to reach only a small group of dedicated researchers, then this may be a sensible goal. For other purposes, we must be careful of the tendency in computing to duplicate essentially the same things over and over again in incompatible ways. We've surely all seen the "14 Competing Standards" XKCD.[6] Stephen Kell introduced the term "fragmentation" [8] for a phenomenon that has precedent outside software: the "coordination problem" [2].

The basic metaphor at work is that of a common language, and natural language is perhaps the oldest example. Geographically dispersed peoples, who lack contact via communication technology, must solve common human problems independently. However, many features of the environment are common to all of them: they certainly need to drink water and must have a word for it. A single tightly-bound tribe would need to converge on the same word ("water", in English). The same mission is pursued by another far-away tribe, but there is no pressure for them to arrive at the same word as the first tribe. Even if they wanted to, they have no systematic means to do so because they are so distant. Because the space of possible words is so large, it is highly likely that they will settle on a completely different word ("mizu", in Japanese). Repeat this process for the majority of words that people need.[7] When the two groups later make contact, or if a third party is familiar with both of them, there will be a sense of artificial choice, or accidental complexity: which language should we use? Words are arbitrary conventions,[8] and collaboration between multiple agents is only possible on the basis of *shared* conventions.

This clashing of conventions occurs with many other things in the physical world: units of measure (such as weight, length, temperature, and money), electrical voltages and plug types, rail track gauges for trains, and whether cars drive on the left or the right. The software world appears to rely even more intensely on this dynamic; my best guess is because (a) computers demand vastly greater precision than fuzzy, ambiguous human perception, and because (b) we must often agree on numbers instead of names. Given two English speakers, it is often plausible that they will happen to use the same English word for a field name, but expecting them to independently hit on the same number (e.g. an enum index, a port, a byte offset) is a far poorer bet.

Alan Kay suggested that most advances in computing (which I will interpret as advances towards *human-friendliness*) were made on the back of late-binding technologies such as the MMU. I'm quite the cheerleader for late-binding, but I think that the victory of standards (often, de-facto) has been just as crucial. This is hard to notice, because we have no reason to think about the competing standards that predate our programming careers.

For example, I have grown up in a world of Unicode (and UTF-8 at that), so I have only rarely needed to convert from other encodings or diagnose bugs originating from mismatches. However, I understand that a lot of programmer hours used to be spent on such things, when there were many character encodings with no clear winner. I am grateful for the de-facto victory of Unicode and UTF-8, and I doubt that much of value was lost as a result. In an alternate history where some competitor to Unicode had won instead, I think I would be similarly grateful (unless it had major flaws incomparable to Unicode's flaws). There cannot be that much virtue in any *particular* scheme for assigning human concepts to numbers; what matters is that we all use the same scheme.

Similarly, while I am aware of the flaws of QWERTY, I am grateful for the fact that there is one main keyboard layout that I need to be familiar with. The more I read about the history of the field, the more grateful I am to live under the winners of past battles: 8-bit bytes, the IBM PC architecture, BIOS, USB, the IP protocol, IEEE 754 floating-point, HTML5, WebGL, `stdin/stdout`, keyboard shortcuts, and so on.

I think the existence of multiple competing conventions is the most wasteful when:

1. Their domain is close to the machine, hence subject to objective engineering criteria rather than high-level human preferences or aesthetics.

[6] https://xkcd.com/927

[7] Of course, people adapt to their local circumstances and their languages follow this. There are untranslatable terms for things that only exist in a particular culture. But this is a higher-order term; there are a great many concepts that are common to many communities: food, water, life, death, etc.

[8] Barring exceptions like onomatopoeia.

2. They have equal size, power, market- or mind-share.

For (1), people can have different high-level preferences, beliefs, or goals. Disagreement and negotiation is legitimate on this basis. However, a clash of *conventions* is different: none of them have any particular claim to objective merit over the others. Each one is simply vying to become the standard that everybody uses. I would expect a group of competing *low-level* conventions to have long resolved the most legitimate engineering arguments, leaving merely different numbering schemes or something similar as the thing being fought over. As for (2), when one convention is much "bigger" (more standard) than the other, there is a clear winner and everyone can see it; its size functions as a natural attractor that could put an end to the war before too long. When there are many competitors with no decisive advantage, one is hesitant to commit to any of the technologies; N-1 out of the N will be the losers at some future point, which is not a promising investment.

I wonder, during that critical period before a decisive winner emerges, what forces end up pushing a convention past the critical point? What are the criteria of natural selection for conventions among human minds following their whims? In the past, governments have simply force-pushed a single standard (measures, money, driving side) on their citizens [9], but they still had to somehow *choose* one standard to force-push instead of the others that were floating around. In software, large companies and communities can succeed at forcing standards in a similar way. I suppose I am more interested in what happens "organically" *before* such an imposition occurs, and even before debate (since, after all, it is only a minority of each camp who debate each other, and even fewer who change their minds as a result): everybody follows their own incentives, a tipping point is reached, and then the losers have to adopt the emergent winner. What are the laws of such a dynamical system? I would like to see "convention-ology" studied (possibly with a better name) and for substrate enthusiasts to show that they are aware of the problems it presents.

(I've blathered enough about this; the online writer Gwern has a very detailed essay on this topic [2] which might make a better jumping-off point for its study.)

7 Graphs of Dictionaries

It is remarkable how many encodings there are for the same basic recurring structure of "graph of key-value dictionaries". C structs, Python objects (and dicts), JS objects, Smalltalk objects ... they're all the same basic *conceptual interface*, augmented on top with slightly different rules for name inheritance, encoded in very different arrangements of bytes. I will even include filesystem directories as another instance, albeit one with the official blessing of the operating system.

This represents a significant chunk of the fragmentation Kell observes under Unix [8]. By prescribing a common language (syscalls and the filesystem) only for "large objects" (files and processes), and remaining silent on how to structure or name the byte sequences inside, Unix ceded the software landscape to many nucleation sites for encoding conventions. Each of these grew independently and evolved its own take on the "graph of dictionaries" pattern, among other things (e.g. tooling and package management).

I would like to see substrates compared by how they differ from this basic common structure. Alternatively, it could be the case that most substrates share this underlying data model yet differ meaningfully in their authoring interfaces. Then again, I'm struggling to fit the spreadsheet into this model; it hardly seems like a graph of dictionaries internally or as a grid interface. Perhaps there are only three families of substrates: those based on dictionary graphs, those restricted to trees, or those based on N-dimensional arrays (flat character lists, vs. 2D grids, vs. 3D grids ...)

8 Research Vision

To summarise what I suppose I've been hinting at throughout this piece, my current research interest is to follow Kell and bridge between those myriad fragmented conventions in the Unix hegemony of programming.

The Unixey substrate consists of a unified "large scope" (the directory tree, effectively authorable) and a million "small scopes" (binary and text file formats, binary formats of heap/stack data; not effectively authorable, or not entirely). Loader code parses binary data into directory-like trees. Parsers

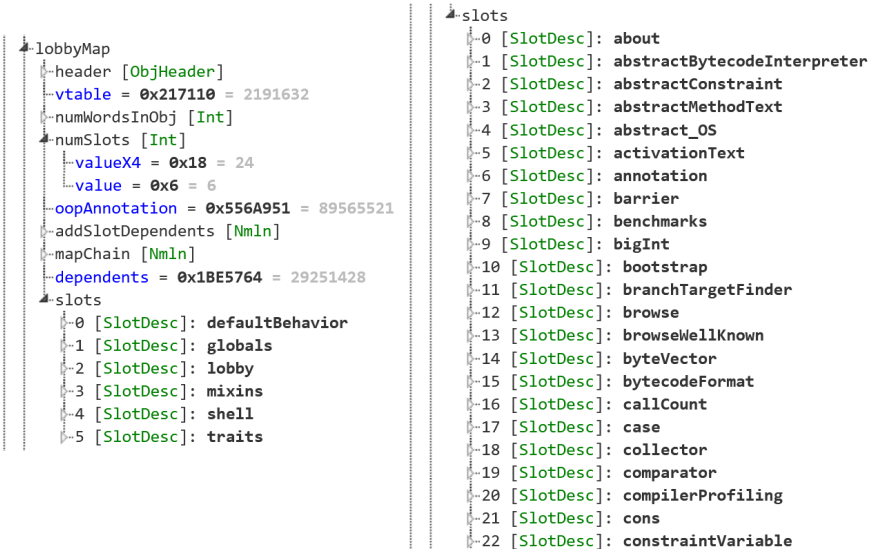

Figure 2: Tree view of Self object slots in the Kaitai online IDE (https://ide.kaitai.io/devel/), parsed on demand from bytes in a Self image file. On the left is the root "lobby" object with its six slots. On the right is the long alphabetical list of all the slot names under `globals`. This is very good for me because I am daunted by the Self VM setup process.

turn textual syntaxes into more of those trees; it's all trees and graphs in the end. Kaitai Struct unfragments the "target language" part for binary data (parse these bytes into C structs, a Python tree, a JS tree, etc). The ubiquitous substrate of the Filesystem, atop appropriate translation layers like Kaitai, could thus undo much of the fragmentation of binary file formats and binary heap data, in tandem with Kell's liballocs [7]. In other words: expose the internal structure of "files" as just more directory trees [11], and shrink the "file" concept to uncontroversially primitive data (individual numbers, names, booleans, and so on).

I'm currently working on Kaitai descriptions of the Squeak Smalltalk[9] and Self VM[10] image formats (Figure 2). By eventually exposing their internal structure as a directory tree via FUSE, I'm optimistic that their embedded object graphs can be made browseable (and even authorable, to an extent) without relying on a running VM process. This approach could assimilate the "black boxes" of binary substrates (and even textual substrates, as unparsed character lists) under a single tree/graph view of *everything*.

[9] https://github.com/jdjakub/kaitai_struct_formats/blob/master/squeak_spur_image64.ksy
[10] https://github.com/jdjakub/kaitai_struct_formats/blob/master/self_2017_uncompressed_snap_32bit.ksy

References

[1] Scott Alexander. *Beware Trivial Inconveniences*. 2009. URL: https://www.lesswrong.com/posts/reitXJgJXFzKpdKyd/beware-trivial-inconveniences.

[2] Gwern Branwen. *Technology Holy Wars are Coordination Problems*. 2020. URL: https://gwern.net/holy-war.

[3] Craig Chambers, David Ungar, and Elgin Lee. "An efficient implementation of SELF a dynamically-typed object-oriented language based on prototypes". In: *ACM Sigplan Notices* 24.10 (1989), pp. 49–70.

[4] Jennifer Coopersmith. *The lazy universe: An introduction to the principle of least action*. Oxford University Press, 2017.

[5] Richard P Gabriel. "Worse is better". In: (1990). URL: http://www.dreamsongs.com/WorseIsBetter.html.

[6] Christopher Kyle Hall. *A New Human-Readability Infrastructure for Computing*. University of California, Santa Barbara, 2017.

[7] Stephen Kell. "Towards a dynamic object model within Unix processes". In: *2015 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)* 2015, pp. 224–239.

[8] Stephen Kell. "Unix, Plan 9 and the Lurking Smalltalk". In: *Reflections on Programming Systems: Historical and Philosophical Aspects* (2018), pp. 189–213.

[9] James C Scott. *Seeing like a state: How certain schemes to improve the human condition have failed*. Yale University Press, 2020.

[10] Gerald Jay Sussman and Jack Wisdom. *Structure and interpretation of classical mechanics*. The MIT Press, 2015.

[11] Raphael Wimmer. "Files as directories: some thoughts on accessing structured data within files". In: *Companion Proceedings of the 2nd International Conference on the Art, Science, and Engineering of Programming*. 2018, pp. 166–170.

********** PAPER 6 **********
AUTHORS: Joel Jakubovic
TITLE: Substrates Vision Statement

++++++++++ REVIEW 1 (Gilad Bracha) +++++++++
I found this essay very hard to follow. I wish it were more clear and concise. Some comments on what I did get out of it.

a. I'm not with Kell. I don't see how those walls are broken, or why I'd enjoy using the result.
The "superficially incomptatible implementations of the key/value dictionary graph" are different languages with different semantics. Some are clean and elegant; some are charitably described as clunky and complex. Trivializing these differences makes no sense to me.

b. Turing completeness is not necessary for a PL (see APL). And it isn't apoint of distinction between PLs and substrates.
c. Substrate is not an alias for platform.
d. I think everyone here understands the issues with "worse is better", adoption, etc. In other words "conventionology" belongs elsewhere. The same can be said about discussions about representation. We know how files work.

There's a real debate to be had how to balance purity and pragmatics (and why purity can be very pragmatic) for concrete reasons, but not in the abstract.

e. Ultra-convenience is good. Zero-install is good. Using JS is bad, even if it sells.
f. I did not find setting up Self to be a problem. But YMMV.
g. AI is the big issue that you haven't discussed.

++++++++++ REVIEW 3 (Jonathan Edwards) +++++++++
I appreciated the call to learn from the history of software and I hope that it informs our discussion at the workshop. There are some valuable nuggets in the summary section:

"Respect Worse-is-Better Darwinism."

"Whether the extinctions of nice substrates like HyperCard were just unlucky, or robust to historical perturbations. If the latter, we should be careful not to make the same mistakes."

"Discover the genuine design tradeoffs rather than cheering “simplicity” etc. On the other hand, identify low-hanging fruit with an argument for why it’s not yet been picked."

This is not a formal review but the start of a discussion so I want to challenge the proposal that "graphs of dictionaries" (AKA an object heap) are a candidate substrate. They are indeed a ubiquitous data model but they carry the covert assumption that the code is static. Programs start with an empty heap. But we want to live in a substrate while we are programming. We can see the problem in a Smalltalk image. Editing a class definition requires all its instances be altered to match. That is not a primitive editing operation on a graph of dictionaries. It gets even worse when we consider that the image contains closures on methods in those class definitions. Editing classes and methods must ensure those closures do something sensible and don't crash. Such issues lead me to disagree with the statement:

"I am inclined to see the ontology of state (“what is state made of?”) as primary, and the “language of evolution” as of secondary importance. Given an ontology of state, primitive edits to that state fall out quite easily, and an infinite variety of programming languages can be dreamt up which compile down to those primitive edits."

I believe the "language of evolution" is indeed a primary problem, although a "high-hanging fruit" that we have found it expedient to ignore. I look forward to continuing this fight in Prague :).

Setting aside that personal disagreement, I appreciate that this proposal brings a distinct perspective to the workshop. I get the impression that Joel is emphasizing issues of virtual machine design.

++++++++++ REVIEW 4 (Tom Larkworthy) +++++++++
The author offers refreshingly strong and clear definitions. A substrate is " Substrate = Primitives + Compositions + Editor + API" and a taxonomy. They also baulk at subjective success criteria, such as user-friendliness. I wonder if that's because they cannot see a path to scientific measurement, however, in industrial contexts there are methodologies to measure this, and AI operated computers can even automate them now.
A contradiction occurs with the Ultra-convenience chapter. To a certain extent, convenience is subjective. Furthermore, the author demands a high quality bar. Unfortunately high quality software is expensive to produce, and requires iterative "subjective" measurement. I fear the quality bar the author wants is exactly the easy-to-use criteria they want to distance themselves from, and unfortunately research software is not the ideal place to produce such software.
Luckily the bulk of the paper outlines a fascinating model for layering a low-level substrate over existing primitives. I think it's quite exciting to come up with a minimal substrate to define the boundary of the substrate concept. Graph of dictionaries is suggested. I personally think lists are important, as dictionaries + lists are the collection types in JSON, and seem fundamental as well as widely supported, perhaps not minimal though.
The dangers of graphs are the same as Turing computability. They are universal representations, same as an array of bytes. So it's semi trivial to say something can be represented as a graph. There is something more to have a parsimonious representation than, it can be represented as. The author hints at this too with "conceptual interface". I think these measures can be formalized further, e.g. a substrate should be total, reversible and a good (low level) substrate will not add much overhead vs. the underlying representation, perhaps?

As a side note I think the https://canvasprotocol.org/ might be an interesting visual representation for the kind of substrate the author intends to build.

++++++++++ REVIEW 5 (Camille Gobert) +++++++++
This statement presents a lot of thoughts related to substrates and their implementation. In particular, it includes a rich discussion of what is and isn't a substrate, according to the author, as well as many examples that are situated within a taxonomy of strategies for organising and constraining information within memory segments (such as file formats and runtime states). It then presents the author's own interest in reducing fragmentation across substrates by better connecting them together and/or exposing one substrate as per another substrate's vocabulary, such as by mounting the graph of objects contained within a Self or Squeak image file as a filesystem.

I really like the depth of this submission and really connect with the goal of reducing fragmentation in computing (also mentioned in Stephen Kell's statement). Moreover, I share this interest in using the file system as a highly standard way of exposing information that is usually stored within application-specific substrates. Following what Joel says in the last paragraph of their statement, I think we should further discuss what would be needed to go from "viewing" information containing in a substrate we don't really control to "authoring" it, which echoes some questions I attempt to raise about (lack of) reactivity in my own statement.

Reading sections 2 and 3 also made we wonder what exactly is, e.g., a PNG substrate: is it the persistent data format that PNG files adhere to? What about the internal representations of the image that they encode, which is effectively what image editors work with and what users would have to work with if they were to program it (a key definitional point according to some statements, such as Jonathan Edwards')?

Related to that, Figure 1 contains examples of substrates that may or may not contain "code"—and those that do may contain instructions at very different levels of abstractions, from machine instructions that are readily readable by a CPU to, e.g., text that describe a program in Haskell's syntax. It therefore seems to me that this view of substrates creates categories: some substrates may only contain logic, some may only contain data, and others may contain both. I very much agree with this view, but it seems to contradict those of other statements, in which combining code and data is a definitional property of a substrate. As such, I think that it would be really helpful to compare our points of views and elaborate on this during the workshop!

Finally, I agree with not optimising for performance first, but I also think that it is important to keep in mind to understand why certain substrates may have succeeded more than others. Might it explain why Smalltalk did not win over Unix back then? This might be related to something raised by Tomas Petricek's statement: the notion of an object(-oriented) substrate has evolved from what Smalltalk introduced to language files that are separate from their runtime instantiations, as in Java, Python, JS and so on.

++++++++++ REVIEW 6 (Antranig Basman) +++++++++
TOTAL SCORE: 2
Overall evaluation: 2 (accept)
Reviewer's confidence: 4 (high)
---- OVERALL EVALUATION ----
Thanks, Joel - a pleasure to read a vision statement that touches with my own on so many points. To start with, great agreement in the first paragraph - the observation that a substrate *might not* harbour a Turing machine whereas a programming language definitely will, is very important. However what is underestimated I think is the effort required to keep the Turing devil out of the kitchen. This is why orienting our work around Kell's notion of an integration domain is so important, since it is one of the very few traditions where the devil might be absent. But what other landmarks can we appeal to in order to steer us away from the tarpits?

For many years I have been fretting over a 2014 posting of Gilad's, "A DOMain of shadows" which I respond to in this blog posting: https://ponder.org.uk/post/2025-04-13-coming-out-of-the-shadows/ - Gilad appears to be presenting us with a dire warning that the tarpit is inescapable in any usable programming context. A key insight I came to a few weeks ago is that pretty much our only other avatar for Turing evasion is the relational algebra of SQL. I think Stephen probably observed this during one of our glory days pub meets but I didn't give it enough attention at the time.

I also agree with the characterisation of "effectively authorable systems" in which every valid configuration of the system is surrounded by a fairly dense neighbourhood of easily characterisable other valid states, to which it can be navigated using forms of self-hosted authoring interfaces.

I agree with this characterisation but not with the reasoning:

Python source code is not an effectively authorable substrate. (Perhaps no Turing-complete programming language is effectively authorable as so defined, because no editor could guarantee the absence of run-time errors. Is Haskell nearly a counterexample?)

I agree that no Turing-complete programming language is effectively authorable, but not because we can't guarantee the absence of run-time errors. Recall that an effective substrate folds together design-time and run-time

- therefore any errors which do occur in an authorable system will appear on the surface of the substrate. Rather than guaranteeing the absence of these, it's our mission to tie their provenance directly to the portion of the substrate which is responsible. Rather, I would say that T-C programming languages are not authorable because of the extreme density of their syntax, and the resulting great distances between valid programs. Robert Hirschfeld's work is very interesting in this regard - his team has worked heroically to produce as much syntax-aware assistance in their "Babylonian programming" systems but I consider that the resulting interfaces will be pretty intimidating to the general audience (see Steve Githen's benchmark of - "we want to be able to target people who aren't quite sure what markdown is"), and they make heavyweight demands on the underlying VM.

So - to the heart of the matter - you correctly ask us to characterise "what are the low-hanging fruit and why haven't they be picked yet". Now I have produced what I consider the basis of a viable substrate, I have to consider that the lowest fruit are not particularly low which is why they haven't been picked. An effective substrate requires strong support from reactive programming primitives which despite good efforts from the RP community (including their 2016 Dagstuhl https://www.dagstuhl.de/seminars/seminar-calendar/seminar-details/16402) are not particularly usable and offer a poor abstraction-cost-to-design-benefit ratio. A key issue is that all the commodity reactive primitives assume that they are embedded in a traditional imperative language and therefore try to get maximum value from traditional assignment/function call semantics which in an effective substrate will not be idiomatic. However, despite this, I've managed to more or less tame a commodity signals implementation (preact-signals) as the underlying engine for my substrate and shelved its terrible ergonomics as something which can be tackled later after the substrate goes through its convention-ology cycle as you interestingly express it.

Other points of agreement - "ultra-convenience" is indeed key. The public will only accept a substrate which can effortlessly follow them wherever they go - including into contexts traditionally considered "static". I found your observation about wrestling with WordPress very interesting since I have just the same problem. Having embedded this visualisation https://biogaliano.org/spiders/ (scroll to bottom) of spider taxonomy using a rather old version of our codebase into my collaborator's WordPress site, I find the resulting integration impossible to navigate. If I want to update this "plugin" I will need to click on endless embeddings of it by hand etc. so it will just need to stay the way it is. It's indeed important that dynamic "snippets" of substrate can be dropped into traditional content using simple textual formats, and that the underlying engine has far lighter runtime machinery than existing near-substrates such as the Lively Kernel, Smalltalk, etc.

So where our views differ I think is in near-term roadmap and priorities. Like you, "I am with Kell" but I am with the "Kell of the future" who has come to enthusiastically adopt web-based technologies, thoroughly enjoys expressing himself in JSON-structured dialects and accepts that every workable substrate needs to expose a self-hosted reactive UI. When we used to meet around the beer taps I would wonder whether Stephen's research programme or mine would bring results in either 50 or 100 years, but now I have a substrate which more or less does what I want I am cutting my own timescales considerably. Whereas I think that a mission to "archeologise the heap" of a traditional programming language will indeed take decades to bring benefits to most users, especially given most of them are not users of traditional programming languages in the first place (you appeal to this problem in your footnote). I would say that the heap memory of every application is a very long way short of half way to being a lingua franca as you are hoping. It's interesting that you in another passage acknowledge the problems of this domain and refer to it as a "failed substrate of bytes in the heap". More generously I would say that it is a kind of "transitional substrate". Without inspiration from this first substrate (via Stephen's Some Were Meant for C) and the second substrate (via Webstrates) we would not have a taste for "being at home amongst state" and have a sense where we were navigating to - the "third substrate" which interestingly enough does seem to follow the progression of gaining a dimension at each stage. I think your Figure 1 also appeals to this - you correctly note that as we progress up the y-axis (and also to the right) we gain increasing constraints on the space of valid states.

Rather than sitting in our lost binary utopia, let's instead make good on Alan Kay's deficit and build on the best substrate that we have - and indeed bundle the "obvious authoring tool" in the browser that it needs. This will still be

far from straightforward since as Jonathan notes, the traditional stack of CSS and JS has become extremely obnoxious and will need a good machete to hack a “bright path” through it of widely intelligible constructs that can be surfaced through our substrate. But it will be easier than we think, I think, with our end goal clearly conceived, and LLMs can be a good help in doing some heavy lifting.

++++++++++ REVIEW 7 (Yann Trividic) +++++++++
The vision statement incrementally builds the notion of substrate, using metaphors from evolutionary biology and languages genetics to explain how we came to have this messy, complicated digital environment, full of incompatible systems. Following Kell, it then proposes leads to glimpse at solutions to unify the various programming convention in Unix in particular.

This is a very clear and accessible article, very well written and well-structured.

The convention-ology concept, the _ontology of state_, and the idea that standards emerge from in various stages, add depth to the more technical parts of the paper. It is not solely about theoretical concepts and technical solutions. It is about how to actually make substrates into adoptable technologies at large.

This part made me reflect on how to achieve agreement on such an ontology, as scaling might raise more difficult, novel qualitative problems, that might not have been met yet in environments such as Kaitai, nor with the semantic web schemas… I need to put more thoughts into this, though.

I found the section on file formats very informative as well. Some formats become _de facto_ substrates because they have been widely adopted and implemented: when edited, or created, they virtually always comply to constraints, which is consistent with Beaudouin-Lafon's take on substrates (Beaudouin-Lafon, 2017). In fact, before writing my own vision paper, I was trying to wrap up my mind around the notion of file formats as substrates. I figured that the "substraty" counterpart of file formats could actually be schema validators... Which is not so far from what the author concurs, in my opinion.

A comment regarding the section about values, on which I may meet a disagreement. Instead of "design[ing] for the software of tomorrow", I believe that we also have a responsibility to design for the hardware of _yesterday_. Ecological stakes make it clear now that we cannot stay on the same technological path we have been on for the past centuries, and building novel technologies represent a cost we might not be able to afford. Though, it is not totally clear what the author was referring to here, and this comment may be irrelevant.

(Same as at least one other reviewer, I prefer signing my reviews, as it is not clear why they should be anonymous in this context!)

++++++++++ REVIEW 8 (Patrick Dubroy) +++++++++
A thought-provoking definition of "substrate"! Many of the other vision statements offered a definition that felt too specific for my tastes. For a new venue (or field), I think the most productive definition is one that casts a wide net, and is "as general as possible, but no general-er". I think your definition — "a set of primitives, and ways to compose them, which is effectively authorable, and has an API" — is definitely in the right ballpark.

I enjoyed the discussion of plain text as a low-level substrate. I actually think that having a few discussions along these lines — is X a substrate or not? — might be a really good way to stimulate some discussion about what aspects we as a group are interested in. Another good one might be: is Git a substrate?

My own take is that substrate is more than "just" a data format; and if plain text and JSON are substrates (although low-level ones) then the definition feels slightly too general to me.

The section "Ultra-convenience" is unexpected, but interesting; sharing meta-tactics (What's the best way to make progress on your ideas? What's the best way to communicate and spread them?) within the group could be fruitful!